SDN-Integrated Multi-Client TCP Chat Application - D1 Design Document

GitHub project: **https://github.com/Abhivbk/CHAT_CLIENT**

- **Abhinav Kulkarni** - chat application: TCP server, client, application protocol, UDP telemetry emitter, and the functional/RTT/scalability test suite.
- **Akshith Singh** - SDN: Ryu controller (learning switch, telemetry listener, priority and spam-throttle policies) and Baseline vs SDN comparison.
- **Amogh B** - Mininet: network topology, bottleneck link, OVS QoS queues, background traffic and experiment automation.

1. What we built

A multi-client chat system over raw TCP sockets, run inside a Mininet network that is controlled by a Ryu (OpenFlow 1.3) SDN controller. The chat server reports what users do (register, join room, send message) to the controller over a separate UDP channel. The controller uses this to give selected chat connections better network treatment on a congested link. We compared the network without this (Baseline) and with it (SDN).

2. Components and how they work

- **Chat server (server.py, Python stdlib only).** One thread per client on TCP port 5000; locks protect the user table, room table and per-client socket writes. Protocol is UTF-8 text, one command per line (max 1024 chars). Commands: /register, /users, /msg, /create, /join, /leave, /rooms, /roommsg, /help, /quit. Users can only join rooms that were created, and empty rooms are deleted.
- **Telemetry (event_emitter.py).** For every state change the server sends one fire-and-forget UDP JSON packet (connect, register, private_msg, room_join, room_leave, room_msg, disconnect) with the client's IP and TCP source port, and optionally appends it to a .jsonl log. Example:
  `{"event":"room_join","user":"alice","room":"vip","ip":"10.0.0.2","port":48122}`
- **Client and tests (client.py, test_clients.py).** Interactive or scripted client. Test runner has three modes: *functional* (pass/fail checks), *rtt* (private-message ack/delivery latency) and *scale* (N clients broadcasting in one room, delivery ratio), with CSV output.
- **Topology (sdn/topology.py).** h1 (server, 10.0.0.1) - s1 ==10 Mbit/s bottleneck== s2 - h2, h3 (clients), h4 (iperf noise). Open vSwitch HTB queues on the bottleneck ports: queue 0 default (min 20%), queue 1 VIP (min 60%, highest priority), queue 2 throttled (max 10%). A virtual port ctl0 (10.0.0.100) lets the server reach the controller.
- **Controller (sdn/sdn_controller.py).** (1) L2 learning switch. (2) A UDP listener thread (port 9999) that maps each event to the TCP 5-tuple of that connection. (3) **Policy A - priority:** when a connection joins a VIP room (vip, load) or registers as a VIP user (rttA, rttB), install flow rules in both directions that use set_queue:1. (4) **Policy B - spam throttle:** more than 40 msgs/s from a user moves their flows to queue 2 for 10 s. Rules are removed on room_leave or disconnect. Mode is selected with SDN_MODE=baseline|sdn.

3. Design decisions

- **Out-of-band UDP telemetry** instead of deep packet inspection: switches cannot read chat commands, and UDP never blocks the server if the controller is down. Trade-off: events can be lost and the controller trusts the server.
- **Per-connection (5-tuple) priority**, because one TCP connection carries all of a user's messages and cannot be split per message. The server therefore reports the client IP:port so rules can match exactly that flow.
- **Threads + blocking sockets** for simplicity and readability at the scale tested (tens of clients); newline framing handles split and merged TCP packets.
- **Same code for both runs:** only SDN_MODE differs, so the comparison isolates the effect of the policies.
- **Queues only on the trunk port**, the only place congestion happens; edge ports use plain output.

4. Experiment and results

Setup: bottleneck 10 Mbit/s; h4 and h1 each send 12 Mbit/s UDP (iperf) across it so the link is saturated both ways. From h2 we run the team's own rtt test (200 messages) and scale test (10 clients x 20 messages), 3 repeats per configuration.

Metric (mean of 3 runs)	Baseline	SDN
Delivery latency avg (ms)	1.95	3.46
Delivery latency p95 (ms)	5.65	12.22
Delivery latency max (ms)	26.1	29.3
Messages lost / delivery ratio (scale)	0 / 1.00	0 / 1.00

Honest reading: no message was lost in either configuration. In two of three SDN runs the p95 delivery latency (2.9 ms and 3.3 ms) was lower than every baseline run (4.2 to 7.4 ms), but the third SDN run was much worse (30.5 ms), which pulls the SDN mean above baseline (std 15.8 ms on p95). With 3 repeats we cannot claim a reliable improvement; SDN-mode switch evidence confirms queue 1 carried the VIP chat traffic while queue 0 carried the iperf flood. Possible causes of the outlier are start-up of the rules (set up after the join event) and VM timing noise. Plots and raw data: sdn/results/comparison.png, comparison.csv, results_*.csv.

5. Build and run

Environment: Ubuntu 22.04 VM with sudo. The chat part needs only Python 3.8+ (no install). The SDN part needs Mininet, Open vSwitch, iperf and Ryu.

A. Chat only, on one machine (1 minute):

```
git clone https://github.com/Abhivbk/CHAT_CLIENT && cd CHAT_CLIENT
python3 server.py                                  # terminal 1
python3 client.py --host 127.0.0.1 --user alice    # terminal 2 (use --user bob in terminal 3)
python3 test_clients.py functional --host 127.0.0.1 --port 5000   # automated checks, server must be running
```

In a client try: `/create vip, /join vip, /roommsg vip hello, /msg bob hi, /help`

B. Full SDN experiment (Mininet + Ryu):

```
cd CHAT_CLIENT/sdn && bash scripts/install_deps.sh     # one time; installs Mininet, OVS, iperf, Ryu (~/ryu-env)
bash scripts/cleanup.sh
# Terminal 1 - controller (use baseline or sdn):
bash scripts/start_controller.sh sdn                # or: baseline
# Terminal 2 - network + automatic experiment (needs ~5-10 min; writes to sdn/results/):
sudo python3 topology.py --experiment sdn --chat-dir ~/CHAT_CLIENT
# Repeat both terminals with baseline (SDN_MODE=baseline / --experiment baseline), then:
python3 compare_results.py                          # makes results/comparison.csv and .png
```

For an interactive demo, run `sudo python3 topology.py` without --experiment, then `xterm h1 h2 h3`: start the server on h1 with `python3 server.py --controller 10.0.0.100:9999` and clients on h2/h3 with `--host 10.0.0.1`. Joining room *vip* makes the controller log "POLICY A" and `bash scripts/show_flows.sh` shows the set_queue rules. If anything is left over, run `bash scripts/cleanup.sh`.

6. Limitations

Priority is per connection, not per message; telemetry is unauthenticated UDP; the experiment has only 3 repeats on a virtual network, so latency numbers are indicative only.